

Windows 开发环境配置指南（从零开始）

本指南面向课程设计组员，从一台全新的 Windows 电脑开始，配置好完整的 C 语言开发环境，能编译运行本项目、使用 Git 协作。

预计耗时：30-60 分钟（取决于网速）

目录

1. [安装 MinGW-w64（GCC 编译器）](#)
2. [安装 CMake](#)
3. [安装 Git](#)
4. [注册 GitHub 账号](#)
5. [安装 VS Code + 插件](#)
6. [Fork 仓库 & 克隆到本地](#)
7. [编译运行项目](#)
8. [Git 日常协作流程](#)
9. [常见问题](#)

1. 安装 MinGW-w64（GCC 编译器）

MinGW-w64 是 Windows 上的 GCC 编译器套件，我们用它来编译 C 代码。

1.1 下载

1. 打开 <https://github.com/nixman/mingw-builds-binaries/releases>
2. 找到最新版本（如 14.2.0 或更高），下载文件名类似：

```
x86_64-14.2.0-release-posix-seh-ucrt-rt_v12-rev2.7z
```

- 如果你的电脑是 64 位（现在几乎都是），选 **x86_64**
- 如果不确定，按 **Win+R** 输入 **msinfo32**，看“系统类型”是否写 x64

提示：如果 GitHub 下载太慢，可以使用镜像站：- <https://mirrors.tuna.tsinghua.edu.cn/github-release/nixman/mingw-builds-binaries/> - 或者搜索 “MinGW-w64 国内下载”

1.2 解压安装

1. 将下载的 `.7z` 文件解压（需要 [7-Zip](#) 或 Bandizip）
2. 把解压出来的 `mingw64` 文件夹放到 **没有中文和空格** 的路径下，推荐：

```
C:\mingw64
```

3. 最终你应该能看到：

```
C:\mingw64\bin\gcc.exe  
C:\mingw64\bin\g++.exe  
C:\mingw64\bin\make.exe
```

1.3 配置环境变量

1. 按 `Win` 键，搜索 “**编辑系统环境变量**”，打开
2. 点击 “**环境变量**” 按钮
3. 在 “**系统变量**”（下半部分）中找到 `Path`，双击编辑
4. 点击 “**新建**”，添加：

```
C:\mingw64\bin
```

5. 一路点 **确定** 保存

1.4 验证安装

新开一个 CMD 或 PowerShell 窗口（重要！旧窗口不会读到新环境变量），输入：

```
gcc --version
```

应该看到类似输出：

```
gcc (x86_64-posix-seh-rev2, Built by MinGW-Builds project) 14.2.0
```

再验证 `make`：

```
mingw32-make --version
```

注意：MinGW-w64 自带的 `make` 叫 `mingw32-make`，不是 `make`。后面会教你怎么配置让 CMake 能正确找到它。

2. 安装 CMake

CMake 是我们的构建系统，负责生成 Makefile。

2.1 下载安装

1. 打开 <https://cmake.org/download/>
2. 下载 **Windows x64 Installer** (如 `cmake-3.xx.x-windows-x86_64.msi`)
3. 运行安装程序，**安装过程中勾选**：
 - ☒ **“Add CMake to the system PATH for all users”** (添加到环境变量，非常重要！)
4. 其他保持默认，一路 Next 完成

2.2 验证

新开 CMD/PowerShell:

```
cmake --version
```

3. 安装 Git

Git 是版本控制工具，用于管理代码和协作。

3.1 下载安装

1. 打开 <https://git-scm.com/download/win>
2. 自动开始下载，运行安装程序
3. 安装过程中大部分保持默认，但注意以下几项：
 - **默认编辑器**：选择 **“Use VS Code as Git’s default editor”** (如果你装了 VS Code)，或选 **“Use Notepad”** 也行
 - **PATH 环境选项**：选 **“Git from the command line and also from 3rd-party software”** (推荐)
 - **行尾转换**：选 **“Checkout as-is, commit as-is”** (这个很重要！避免 Windows/Linux 混用时行尾符出问题)
 - 其余默认即可

3.2 验证

```
git --version
```

3.3 基本配置

安装完后，打开 Git Bash 或 PowerShell，设置你的个人信息（提交代码时会带上）：

```
git config --global user.name "你的名字"
git config --global user.email "你的邮箱@example.com"
```

把引号里的内容换成你自己的。邮箱建议和 GitHub 注册邮箱一致。

4. 注册 GitHub 账号

4.1 注册

1. 打开 <https://github.com/>
2. 点击 **Sign up**
3. 输入邮箱、密码、用户名
4. 完成验证（拖拼图之类的）
5. 去邮箱收验证码，输入确认

4.2 注意事项

- **用户名** 建议用英文小写 + 数字，如 `zhangsan2026`，不要用中文
- **邮箱** 建议用常用邮箱，后续要收通知
- 注册完成后，把你的 **GitHub 用户名** 告诉组长，方便添加为仓库协作者

5. 安装 VS Code + 插件

5.1 安装 VS Code

1. 打开 <https://code.visualstudio.com/>
2. 下载 Windows 版，运行安装
3. 安装时勾选：
 - ☒ “Add to PATH”
 - ☒ “Register Code as an editor for supported file types”
 - ☒ “Add ‘Open with Code’ action to Windows Explorer”

5.2 安装插件

打开 VS Code，点左侧的 **扩展图标**（或按 `Ctrl+Shift+X`），搜索并安装以下插件：

插件名	用途	搜索关键词
C/C++	C 语言智能提示、调试	C/C++ (Microsoft)
C/C++ Extension Pack	CMake 等一整套工具	C/C++ Extension Pack
CMake Tools	CMake 项目管理、一键编译	CMake Tools
Chinese (Simplified)	VS Code 中文界面	Chinese

其中 **C/C++ Extension Pack** 已经包含了 C/C++ 和 CMake Tools，装这一个就够了，它会自动把依赖的都装上。

5.3 配置 VS Code 使用 MinGW

安装完插件后，需要告诉 VS Code 去哪里找 GCC：

- 1. 按 `Ctrl+Shift+P`，输入 “**C/C++: Edit Configurations (UI)**”，回车
- 2. 找到 “**Compiler path**”，填入：

```
C:\mingw64\bin\gcc.exe
```

- 3. “**IntelliSense mode**” 选 `windows-gcc-x64`

5.4 配置 CMake 使用 MinGW Makefiles

- 1. 按 `Ctrl+Shift+P`，输入 “**CMake: Select a Kit**”，回车
- 2. 应该能看到 `GCC 14.2.0 x86_64-w64-mingw32`，选中它
- 3. 如果没看到，点 “**Scan for kits**” 让它重新扫描

注意：在 Windows 上，CMake 默认生成 Visual Studio 项目文件，但我们需要用 MinGW 的 Makefiles。如果 CMake Tools 插件找不到 Kit，你也可以用命令行编译（见第 7 节）。

6. Fork 仓库 & 克隆到本地

6.1 Fork 仓库

Fork 就是在你自己的 GitHub 账号下创建一份仓库的副本，这样你可以在自己的副本上自由修改，不影响原仓库。

- 1. 打开仓库页面：https://github.com/Mel-SRK/Automobile_Sales_Information_Management_System

2. 点击右上角的 **Fork** 按钮
3. 选择你的账号，点 **Create fork**
4. 等待几秒，你会跳转到你自己的仓库副本，URL 变成：

```
https://github.com/你的用户名/Automobile_Sales_Information_Management_System
```

6.2 克隆到本地

1. 在你电脑上选一个放代码的目录，比如：

```
D:\Projects
```

2. 打开 PowerShell 或 Git Bash，进入该目录：

```
cd D:\Projects
```

3. 克隆你的 fork（不是组长的原始仓库）：

```
git clone https://github.com/你的用户名/Automobile_Sales_Information_Management_System.git
```

4. 进入项目目录：

```
cd Automobile_Sales_Information_Management_System
```

5. 添加原始仓库为 **upstream**（上游），方便后续同步组长的更新：

```
git remote add upstream https://github.com/Mel-SRK/Automobile_Sales_Information_Management_System.git
```

6. 验证远程仓库配置：

```
git remote -v
```

应该看到：

```
origin    https://github.com/你的用户名/Automobile_Sales_Information_Management_System.git (fetch)
origin    https://github.com/你的用户名/Automobile_Sales_Information_Management_System.git (push)
upstream  https://github.com/Mel-SRK/Automobile_Sales_Information_Management_System.git (fetch)
upstream  https://github.com/Mel-SRK/Automobile_Sales_Information_Management_System.git (push)
```

6.3 用 VS Code 打开项目

```
code .
```

或者在 VS Code 中：文件 → 打开文件夹 → 选择项目目录。

7. 编译运行项目

方法一：用命令行编译（推荐新手先用这个，最直观）

在项目根目录打开 PowerShell：

```
# 创建 build 目录并进入
mkdir build
cd build

# 生成 Makefile (使用 MinGW)
cmake .. -G "MinGW Makefiles"

# 编译
mingw32-make

# 运行
.\auto_sales.exe
```

 注意：Windows 上 CMake 必须指定 `-G "MinGW Makefiles"`，否则它会尝试找 Visual Studio。

清理重新编译：

```
# 在项目根目录执行
Remove-Item -Recurse -Force build
mkdir build; cd build
cmake .. -G "MinGW Makefiles"
mingw32-make
.\auto_sales.exe
```

方法二：用 VS Code 的 CMake Tools 插件

1. 用 VS Code 打开项目文件夹
2. 底部状态栏应该出现 CMake 相关按钮
3. 点击 **Build**（或按 `F7`）编译
4. 点击 **Run**（或按 `Shift+F5`）运行

如果 CMake Tools 报错找不到 Generator，按 `Ctrl+Shift+P` → “CMake: Edit User-Local CMake Kits”，确保配置了 MinGW。

8. Git 日常协作流程

8.1 基本概念

你的电脑 (local) $\longleftrightarrow$ 你的 GitHub (origin) $\longleftrightarrow$ 组长的 GitHub (upstream)
编码 push/fetch fetch + Pull Request

- **local**: 你电脑上的代码
- **origin**: 你的 GitHub fork
- **upstream**: 组长的原始仓库

8.2 日常工作流程

第一步：同步组长的最新代码

每次开始写代码前，先拉取组长的最新更新：

```
# 切换到 main 分支
git checkout main

# 拉取组长的最新代码
git fetch upstream

# 合并到你的 main
git merge upstream/main

# 推送到你的 GitHub
git push origin main
```

第二步：创建功能分支

不要直接在 main 上改代码！ 为每个功能创建一个新分支：

```
# 创建并切换到新分支
git checkout -b 你的名字/功能名
# 例如：
git checkout -b zhangsan/employee-module
```

第三步：写代码、编译测试

用 VS Code 写代码，用命令行或 CMake Tools 编译测试。

第四步：提交代码


```
# 查看你改了哪些文件
git status

# 添加所有修改
git add .

# 提交（写清楚你做了什么）
git commit -m "feat: 实现员工信息模块的增删改查"
```

提交信息规范： - feat: 新功能 - fix: 修复 bug - docs: 文档修改 - 不用英文也行，写清楚就好，如 `git commit -m "实现员工添加功能"`

第五步：推送到你的 GitHub

```
git push origin 你的名字/功能名
```

第六步：创建 Pull Request (PR)

1. 打开你的 GitHub fork 页面
2. 应该会看到一个提示 “**Compare & pull request**”，点击它
3. 或者手动操作：
 - 点击 **Pull requests** → **New pull request**
 - Base 选组长的 `main`，Compare 选你的功能分支
4. 填写标题和描述（说明你做了什么）
5. 点击 **Create pull request**

组长审核后会合并到主仓库。

8.3 完整示例

假设你要实现 `employee.c`：

```
# 1. 同步最新代码
git checkout main
git fetch upstream && git merge upstream/main && git push origin main

# 2. 创建分支
git checkout -b zhangsan/employee

# 3. 写代码... (用 VS Code)
# ...

# 4. 编译测试
cd build
mingw32-make
```

```
.\auto_sales.exe # 测试一下功能是否正常
cd ..

# 5. 提交
git add .
git commit -m "feat: 实现员工模块完整功能（增删改查列表）"

# 6. 推送
git push origin zhangsan/employee

# 7. 去 GitHub 创建 PR
```

9. 常见问题

Q: cmake 提示找不到 C 编译器

```
CMake Error: No CMAKE_C_COMPILER could be found.
```

原因：CMake 没找到 GCC。检查：1. `gcc --version` 能不能正常输出 2. `C:\mingw64\bin` 有没有加到系统 PATH 3. 修改 PATH 后有没有开新终端（旧终端不会读到新 PATH）

Q: cmake .. 报错说找不到 Generator

```
CMake Error: Could not create named generator MinGW Makefiles
```

原因：可能是用了 Visual Studio 的 CMake 而不是 MinGW 的。试试用完整路径：

```
C:\mingw64\bin\cmake.exe .. -G "MinGW Makefiles"
```

或者确保 PATH 中 MinGW 的 `bin` 目录在 Visual Studio 之前。

Q: make 命令不存在

MinGW-w64 自带的叫 `mingw32-make`，不是 `make`。你有两个选择：

方案 A：直接用 `mingw32-make`

方案 B（推荐）：创建一个 `make` 的快捷方式

```
cd C:\mingw64\bin
copy mingw32-make.exe make.exe
```

这样以后直接用 `make` 就行了。

Q: `git clone` 太慢 / 连不上 GitHub

方案一：使用代理（如果你有的话）

方案二：使用镜像

```
# 用 ghproxy 镜像克隆
git clone https://ghproxy.com/https://github.com/你的用户名/Automobile_Sales_Information_Management_System.git
```

方案三：下载 ZIP 直接在 GitHub 仓库页面点 **Code** → **Download ZIP**，但这样不能用 Git 推送代码，后续还需要配置。

Q: VS Code 里 `#include` 报错（红色波浪线）

1. 按 `Ctrl+Shift+P`
2. 输入 “**C/C++: Edit Configurations (UI)**”
3. 确认 **Compiler path** 是 `C:\mingw64\bin\gcc.exe`
4. 确认 **IntelliSense mode** 是 `windows-gcc-x64`

Q: 编译成功但运行时中文乱码

Windows CMD 默认编码不是 UTF-8。解决方法：

方案一（推荐）：在 PowerShell 中运行前先执行：

```
chcp 65001
.\auto_sales.exe
```

方案二：在 Windows Terminal 中运行（Windows 11 自带），它默认支持 UTF-8。

方案三：在代码中不依赖系统终端编码，使用英文菜单（最后的方案）。

Q: 怎么撤销还没提交的修改？

```
# 撤销单个文件
git checkout -- 文件名

# 撤销所有修改（危险！确认没改错东西再执行）
git checkout -- .
```

Q: 怎么查看提交历史？

```
git log --oneline
```

Q: 不小心在 main 分支上改了代码怎么办？

```
# 先把修改暂存
git stash

# 创建新分支
git checkout -b 你的名字/功能名

# 恢复修改
git stash pop
```

附录：一行命令速查

操作	命令
编译项目	cd build && cmake .. -G "MinGW Makefiles" && mingw32-make
运行程序	.\auto_sales.exe
查看状态	git status
提交修改	git add . && git commit -m "描述"
推送代码	git push origin 分支名
同步组长代码	git fetch upstream && git merge upstream/main
查看历史	git log --oneline

有问题找组长！ 不确定的操作先问，别怕问蠢问题，搞坏了代码才真的麻烦。